

LAPORAN PRAKTIKUM

STRUKTUR DATA

Insertion, Selection, dan Bubble Sort

disusun Oleh:

RIFKI MAULANA

NIM 2511533007

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM : ZAHARAN AZARIA ANVAYA



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 21 MEI 2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul “**Insertion, Selection, dan Bubble Sort**”. Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai cara pemrograman struktur data Insertion, Selection, dan Bubble Sort.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 21 Mei 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM.....	1
KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Kode Program	2
2.1.1 InsertionSort	2
2.1.2 SelectionSort.....	2
2.1.3 BubleSort	3
2.1.4 InsertionSortGUI	5
BAB III KESIMPULAN	9
3.1 Kesimpulan.....	9
DAFTAR PUSTAKA.....	10
BAB IV Laporan Tugas.....	11

BAB I

PENDAHULUAN

1.1 Latar Belakang

Algoritma insertion sort, adalah metode pengurutan dengan cara menyisipkan elemen data pada posisi yang tepat. Pencarian posisi yang tepat dilakukan dengan melakukan pencarian berurutan didalam barisan elemen, selama pencarian posisi yang tepat dilakukan pergeseran elemen. Selection sort merupakan metode pengurutan dengan mencari nilai data terkecil dimulai dari data diposisi 0 hingga diposisi N-1. Bubble Sort adalah algoritma pengurutan paling sederhana yang bekerja dengan membandingkan dua elemen yang berdekatan, lalu menukarnya jika posisinya salah. Proses ini diulang sampai tidak ada lagi pertukaran yang terjadi.

1.2 Tujuan Praktikum

- Memahami konsep Sorting Insertion, selection, dan Bubble dalam pemrograman.
- Mampu merancang dan membuat GUI dari sorting Insertion, selection, dan Bubble pada java.

1.3 Manfaat Praktikum

- Menambah pemahaman mengenai penerapan Sorting dalam kasus kehidupan nyata.
- Melatih kemampuan membuat program yang terstruktur.
- Membantu memahami penggunaan class sebagai representasi objek di Java.

BAB II PEMBAHASAN

2.1 Kode Program

2.1.1 InsertionSort

```
package pekan7_2511533007;

public class InsertionSort_2511533007 {
    public static void insertionSort_3007 (int[] arr_3007) {
        int n_3007 = arr_3007.length;
        for (int i_3007 = 1; i_3007 < n_3007; i_3007++) {
            int key_3007 = arr_3007[i_3007];
            int j_3007 = i_3007 - 1;
            while (j_3007 >= 0 && arr_3007[j_3007] > key_3007) {
                arr_3007[j_3007 + 1] = arr_3007[j_3007];
                j_3007--;
            }
            arr_3007[j_3007 + 1] = key_3007;
        }
    }

    public static void main(String[] args) {
        int arr_3007[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3007 = arr_3007.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
        insertionSort_3007(arr_3007);
        System.out.printf("array yang terurut:\n");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
    }
}
```

Kode Program 2.1

Class ini berfungsi untuk mengimplementasikan algoritma Insertion dengan cara mengurutkan array angka dari nilai terkecil ke terbesar. Di dalam class ini, method insertionSort_3007 memproses pengurutan dengan cara mengambil satu elemen array sebagai kunci, membandingkannya dengan elemen elemen di sebelah kirinya yang sudah terurut, lalu menggeser elemen yang lebih besar ke kanan sebelum menyisipkan elemen kunci tersebut ke posisi yang tepat. Sementara itu, method main bertindak sebagai titik masuk program yang mendeklarasikan array angka acak, menampilkan kondisi awal array ke layar, memanggil fungsi pengurutan, dan mencetak kembali hasil akhir array yang telah rapi.

```
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78
```

2.1.2 SelectionSort

```
package pekan7_2511533007;

public class SelectionSort_2511533007 {
    public static void selectionSort_3007 (int[] arr_3007) {
```

```

        int n_3007 = arr_3007.length;
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++) {
            int minIndex_3007 = i_3007;
            for (int j_3007 = i_3007 + 1; j_3007 < n_3007; j_3007++) {
                if (arr_3007[j_3007] < arr_3007[minIndex_3007]) {
                    minIndex_3007 = j_3007;
                }
            }
            int temp_3007 = arr_3007[i_3007];
            arr_3007[i_3007] = arr_3007[minIndex_3007];
            arr_3007[minIndex_3007] = temp_3007;
        }
    }
    public static void main(String[] args) {
        int arr_3007[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3007 = arr_3007.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
        selectionSort_3007(arr_3007);
        System.out.printf("array yang terurut:\n");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
    }
}

```

Kode Program 2.2

Class ini mengimplementasikan algoritma Selection Sort untuk mengurutkan data angka di dalam array. Di dalamnya terdapat method `selectionSort_3007` yang bekerja dengan cara memindai array dari indeks awal untuk mencari elemen dengan nilai paling minimum di sisa deretan array yang belum terurut, kemudian menukarnya dengan elemen pada posisi indeks acuan saat itu. Selanjutnya, method `main` bertugas untuk menyiapkan data array mentah, menampilkan deretan data sebelum diproses, mengeksekusi fungsi pengurutan tersebut, dan menampilkan hasil akhir array yang sudah terurut ke layar.

```

array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

2.1.3 BubleSort

```

package pekan7_2511533007;

public class BubleSort_2511533007 {
    public static void bubbleSort_3007(int[] arr_3007) {
        int n_3007 = arr_3007.length;
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++) {
            for (int j_3007 = 0; j_3007 < n_3007 - i_3007 - 1; j_3007++) {
                if (arr_3007[j_3007] > arr_3007[j_3007 + 1]) {
                    int temp_3007 = arr_3007[j_3007];
                    arr_3007[j_3007] = arr_3007[j_3007 + 1];
                    arr_3007[j_3007 + 1] = temp_3007;
                }
                // System.out.println("data:" + arr[j] + " " + arr[j + 1]);
            }
        }
    }
}

```

```

    }

    public static void main(String[] args) {
        int arr_3007[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3007 = arr_3007.length;
        System.out.print("array yang belum terurut:");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
        bubbleSort_3007(arr_3007);
        System.out.print("array yang terurut menggunakan BubleSort:");
        for (int i_3007 = 0; i_3007 < n_3007; i_3007++)
            System.out.print(arr_3007[i_3007] + " ");
        System.out.println("");
    }
}

```

Kode Program 2.3

Class ini menerapkan algoritma Bubble Sort untuk mengurutkan array angka dengan cara membandingkan elemen berpasangan secara terus menerus. Method bubbleSort_3007 di dalam class ini mengurutkan data menggunakan dua perulangan dengan membandingkan dua elemen yang bersebelahan, di mana jika elemen kiri lebih besar dari elemen kanan maka posisinya akan ditukar sehingga nilai terbesar perlahan bergeser ke ujung kanan array. Untuk menjalankan proses ini, method main berfungsi menyiapkan data array mentah, mencetaknya ke layar, mengeksekusi fungsi bubble sort, dan menampilkan visualisasi teks array yang telah berhasil diurutkan.

```

array yang belum terurut:23 78 45 8 32 56 1
array yang terurut menggunakan BubleSort:23 78 78 78 78 78 78

```

2.1.4 InsertionSortGUI

```
1 package pekan7_2511533007;
2
3 import java.awt.BorderLayout;
4
22
23 public class InsertionSortGUI_2511533007 extends JFrame {
24
25     private static final long serialVersionUID = 1L;
26     private JPanel contentPane;
27     private int[] array_3007;
28     private JLabel[] labelArray_3007;
29     private JButton stepButton_3007, resetButton_3007, setButton_3007;
30     private JTextField inputField_3007;
31     private JPanel panelArray_3007;
32     private JTextArea stepArea_3007;
33
34     private int i_3007 = 1, j_3007;
35     private boolean sorting_3007 = false;
36     private int stepCount_3007 = 1;
37
38     public InsertionSortGUI_2511533007() {
39         setTitle("Insertion Sort Langkah per Langkah");
40         setSize(750, 400);
41         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42         setLocationRelativeTo(null);
43         setLayout(new BorderLayout());
44
45         // Panel input
46         JPanel inputPanel_3007 = new JPanel(new FlowLayout());
47         inputField_3007 = new JTextField(30);
48         setButton_3007 = new JButton("Set Array");
49         inputPanel_3007.add(new JLabel ("Masukkan angka (pisahkan dengan koma:"));
50         inputPanel_3007.add(inputField_3007);
51         inputPanel_3007.add(setButton_3007);
52
53         // Panel array visual
54         panelArray_3007 = new JPanel();
55         panelArray_3007.setLayout(new FlowLayout());
56
57         // Panel kontrol
58         JPanel controlPanel_3007 = new JPanel();
59         stepButton_3007 = new JButton("Langkah Selanjutnya");
60         resetButton_3007 = new JButton("Reset");
61         stepButton_3007.setEnabled(false);
62         controlPanel_3007.add(stepButton_3007);
63         controlPanel_3007.add(resetButton_3007);
64
65         // Area teks untuk log langkah-langkah
66         stepArea_3007 = new JTextArea(8, 60);
67         stepArea_3007.setEditable(false);
68         stepArea_3007.setFont(new Font("Monospaced", Font.PLAIN, 14));
69         JScrollPane scrollPane_3007 = new JScrollPane(stepArea_3007);
70
71         // Tambahkan panel ke frame
72         add(inputPanel_3007, BorderLayout.NORTH);
73         add(panelArray_3007, BorderLayout.CENTER);
74         add(controlPanel_3007, BorderLayout.SOUTH);
75         add(scrollPane_3007, BorderLayout.EAST);
76
77         // Event Set Array
78         setButton_3007.addActionListener(e -> setArrayFromInput());
79
80         // Event Langkah Selanjutnya
81         stepButton_3007.addActionListener(e -> performStep_3007());
82
83         // Event Reset
84         resetButton_3007.addActionListener(e -> reset_3007());
85     }
```



```

86
87 private void setArrayFromInput() {
88     String text_3007 = inputField_3007.getText().trim();
89     if (text_3007.isEmpty()) return;
90     String [] parts_3007 = text_3007.split(",");
91     array_3007 = new int[parts_3007.length];
92     try {
93         for (int k_3007 = 0; k_3007 < parts_3007.length; k_3007++) {
94             array_3007[k_3007] = Integer.parseInt(parts_3007[k_3007].trim());
95         }
96     } catch (NumberFormatException e) {
97         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
98             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
99         return;
100     }
101     i_3007 = 1;
102     stepCount_3007 = 1;
103     sorting_3007 = true;
104     stepButton_3007.setEnabled(true);
105     stepArea_3007.setText("");
106     panelArray_3007.removeAll();
107     labelArray_3007 = new JLabel[array_3007.length];
108     for (int k_3007 = 0; k_3007 < array_3007.length; k_3007++) {
109         labelArray_3007[k_3007] = new JLabel(String.valueOf(array_3007[k_3007]));
110         labelArray_3007[k_3007].setFont(new Font("Arial", Font.BOLD, 24));
111         labelArray_3007[k_3007].setBorder(BorderFactory.createLineBorder(Color.BLACK));
112         labelArray_3007[k_3007].setPreferredSize(new Dimension(50, 50));
113         labelArray_3007[k_3007].setHorizontalAlignment(SwingConstants.CENTER);
114         panelArray_3007.add(labelArray_3007[k_3007]);
115     }
116     panelArray_3007.revalidate();
117     panelArray_3007.repaint();
118 }
119
120 private void performStep_3007() {
121     if (i_3007 < array_3007.length && sorting_3007) {
122         int key_3007 = array_3007[i_3007];
123         j_3007 = i_3007 - 1;
124
125         StringBuilder stepLog_3007 = new StringBuilder();
126         stepLog_3007.append("Langkah ").append(stepCount_3007).
127             append(": Memasukkan ").append(key_3007).append("\n");
128
129         while (j_3007 >= 0 && array_3007[j_3007] > key_3007) {
130             array_3007[j_3007 + 1] = array_3007[j_3007];
131             j_3007--;
132         }
133         array_3007[j_3007 + 1] = key_3007;
134
135         updateLabels_3007();
136         stepLog_3007.append("Hasil: ").append(arrayToString_3007(array_3007)).append("\n\n");
137         stepArea_3007.append(stepLog_3007.toString());
138
139         i_3007++;
140         stepCount_3007++;
141
142         if (i_3007 == array_3007.length) {
143             sorting_3007 = false;
144             stepButton_3007.setEnabled(false);
145             JOptionPane.showMessageDialog(this, "Sorting selesai!");
146         }
147     }
148 }
149

```

```

150 private void updateLabels_3007() {
151     for (int k_3007 = 0; k_3007 < array_3007.length; k_3007++) {
152         labelArray_3007[k_3007].setText(String.valueOf(array_3007[k_3007]));
153     }
154 }
155
156 private void reset_3007() {
157     inputField_3007.setText("");
158     panelArray_3007.removeAll();
159     panelArray_3007.revalidate();
160     panelArray_3007.repaint();
161     stepArea_3007.setText("");
162     stepButton_3007.setEnabled(false);
163     sorting_3007 = false;
164     i_3007 = 1;
165     stepCount_3007 = 1;
166 }
167
168 private String arrayToString_3007(int[] arr) {
169     StringBuilder sb_3007 = new StringBuilder();
170     for (int k = 0; k < arr.length; k++) {
171         sb_3007.append(arr[k]);
172         if (k < arr.length - 1) sb_3007.append(", ");
173     }
174     return sb_3007.toString();
175 }
176
177 public static void main(String[] args) {
178     SwingUtilities.invokeLater(() -> {
179         InsertionSortGUI_2511533007 gui = new InsertionSortGUI_2511533007();
180         gui.setVisible(true);
181     });
182 }
183 }

```

Kode Program 2.4

Class ini menyediakan visualisasi untuk algoritma Insertion Sort menggunakan GUI berbasis library Java Swing. Konstruktor class ini berfungsi membangun struktur window aplikasi, mengatur tata letak komponen visual, serta menangani fungsi penanganan klik pada setiap tombol.

Method `setArrayFromInput` bertugas mengambil input string angka dari pengguna, memisahkannya berdasarkan koma, mengubahnya menjadi array integer, serta menggambar komponen kotak label di layar.

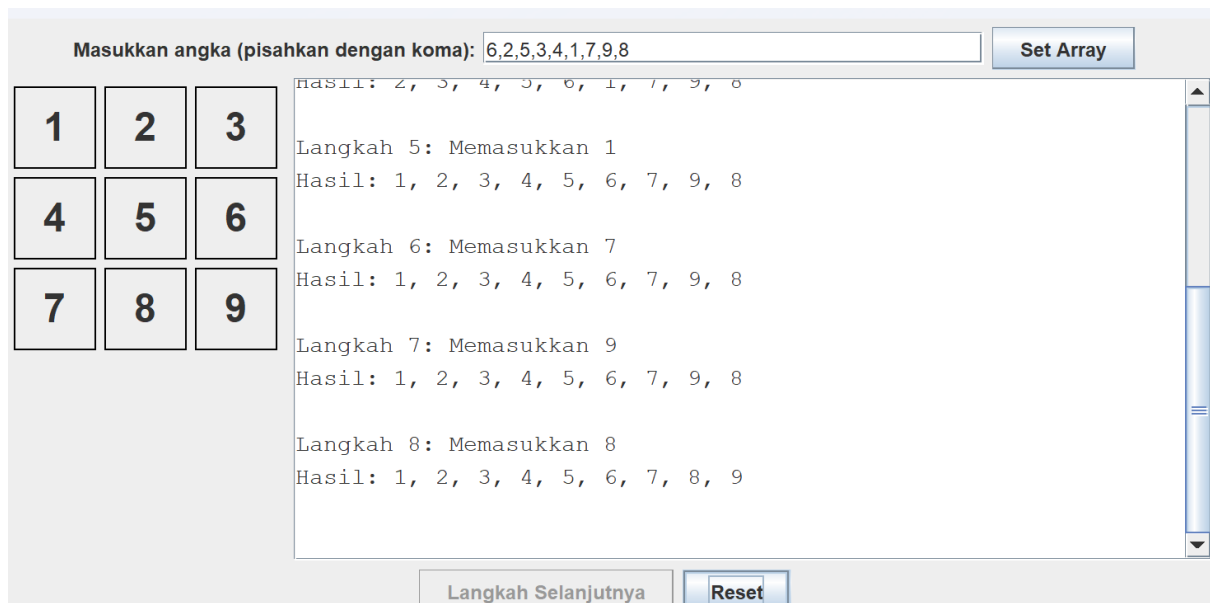
Method `performStep_3007` mengeksekusi satu iterasi luar dari Insertion Sort setiap kali tombol ditekan, melakukan pergeseran nilai di latar belakang, dan mencatat aktivitasnya ke area teks log.

Method `updateLabels_3007` berfungsi memperbarui teks di dalam kotak label visual pada aplikasi agar perubahan posisi angka langsung terlihat oleh pengguna.

Method `reset_3007` digunakan untuk membersihkan kolom input, menghapus visualisasi kotak angka, mengosongkan riwayat log, serta mengembalikan semua status variabel kontrol ke kondisi awal.

Method `arrayToString_3007` merupakan fungsi pembantu untuk mengubah susunan data angka di dalam array menjadi sebaris teks string yang dipisahkan koma agar mudah dibaca pada log.

Terakhir, method `main` berfungsi menjalankan dan menampilkan jendela aplikasi GUI ini secara aman di dalam thread khusus penanganan event Java Swing.



BAB III

KESIMPULAN

3.1 Kesimpulan

Algoritma Insertion Sort, Selection Sort, dan Bubble Sort merupakan algoritma sorting fundamental yang masing-masing memiliki mekanisme unik dalam menata data angka acak secara ascending. Insertion Sort bekerja dengan cara menyisipkan elemen ke posisi yang tepat pada bagian array yang telah terurut secara bertahap, sementara Selection Sort berfokus pada pencarian nilai minimum di setiap iterasi untuk ditukarkan langsung ke indeks acuannya. Di sisi lain, Bubble Sort menerapkan pendekatan yang lebih sederhana dengan membandingkan dan menukar dua elemen yang bersebelahan secara terus-menerus hingga nilai terbesar perlahan bergeser ke ujung akhir array.

Ketiga algoritma ini berhasil membuktikan bahwa meskipun memiliki logika perulangan yang berbeda, ketiganya mampu menghasilkan urutan data yang sama rapi. Selain itu, pengembangan kode keempat yang menggunakan GUI membuktikan bahwa visualisasi langkah demi langkah sangat efektif untuk mempermudah pemahaman mengenai pergerakan, pergeseran, dan penukaran posisi indeks data di dalam memori pada setiap tahapan iterasi. Kode-kode tersebut menunjukkan bahwa keberhasilan manipulasi dan eksekusi algoritma sorting sangat bergantung pada ketepatan kondisi perulangan serta efisiensi penukaran variabel di dalam kode program, sehingga pemahaman alur ini sangat penting untuk menentukan jenis sorting yang paling sesuai dengan kebutuhan pengembangan software.

DAFTAR PUSTAKA

- [1] E. Retnoningsih, "Algoritma Pengurutan Data (Sorting) Dengan Metode Insertion Sort dan Selection Sort," *Information Management For Educators And Professionals*, vol. 3, no. 1, hlm. 95-106, Des. 2018. [Daring]. Tersedia pada: <https://media.neliti.com/media/publications/414701-none-931dcaa.pdf>. [Diakses: 21-Mei-2026].
- [2] BINUS University Bandung, "Algoritma Selection Sort di Python," 24 Oktober 2023. [Daring]. Tersedia pada: <https://binus.ac.id/bandung/2023/10/algoritma-selection-sort-di-python/>. [Diakses: 21-Mei-2026].
- [3] Telkom University Jakarta, "Sorting Algorithm: Bubble, Selection, dan Insertion Sort," 24 November 2023. [Daring]. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/sorting-algorithm-bubble-selection-dan-insertion-sort/>. [Diakses: 21-Mei-2026].

BAB IV Laporan Tugas

A. Kode ADT

```
package pekan7_2511533007;

public class Mahasiswa_2511533007 {
    // Atribut
    private String nama_3007;
    private String nim_3007;
    private String prodi_3007;

    // Konstruktor
    public Mahasiswa_2511533007(String nama_3007, String nim_3007, String prodi_3007) {
        this.nama_3007 = nama_3007;
        this.nim_3007 = nim_3007;
        this.prodi_3007 = prodi_3007;
    }

    // Getter
    public String getNama_3007() {
        return nama_3007;
    }

    public String getNim_3007() {
        return nim_3007;
    }

    public String getProdi_3007() {
        return prodi_3007;
    }

    // Setter
    public void setNama_3007(String nama_3007) {
        this.nama_3007 = nama_3007;
    }

    public void setNim_3007(String nim_3007) {
        this.nim_3007 = nim_3007;
    }

    public void setProdi_3007(String prodi_3007) {
        this.prodi_3007 = prodi_3007;
    }

    @Override
    public String toString() {
        return nama_3007;
    }
}
```

B. Kode Program

```
package pekan7_2511533007;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.GridLayout;
import java.util.ArrayList;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
```

```

import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class GUIMahasiswa_2511533007 extends JFrame {

    private static final long serialVersionUID = 1L;
    private JTextField inputNama_3007, inputNim_3007, inputProdi_3007;
    private JButton btnTambah_3007, btnMulaiSort_3007, btnLangkah_3007, btnReset_3007;
    private JComboBox<String> comboSort_3007;
    private JPanel panelArray_3007;
    private JLabel[] labelArray_3007;
    private JTextArea stepArea_3007;
    private ArrayList<Mahasiswa_2511533007> dataAwal_3007;
    private ArrayList<Mahasiswa_2511533007> dataProses_3007;
    private int i_3007 = 0;
    private int stepCount_3007 = 1;
    private boolean sorting_3007 = false;
    private String tipeSort_3007 = "";

    public GUIMahasiswa_2511533007() {
        setTitle("Visualisasi Sorting Mahasiswa");
        setSize(900, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        dataAwal_3007 = new ArrayList<>();

        JPanel panelUtara_3007 = new JPanel(new GridLayout(4, 2, 5, 5));
        panelUtara_3007.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));

        panelUtara_3007.add(new JLabel(" Nama Mahasiswa:"));
        inputNama_3007 = new JTextField();
        panelUtara_3007.add(inputNama_3007);

        panelUtara_3007.add(new JLabel(" NIM Mahasiswa:"));
        inputNim_3007 = new JTextField();
        panelUtara_3007.add(inputNim_3007);

        panelUtara_3007.add(new JLabel(" Program Studi:"));
        inputProdi_3007 = new JTextField();
        panelUtara_3007.add(inputProdi_3007);

        btnTambah_3007 = new JButton("Tambah Data ke Array");
        panelUtara_3007.add(new JLabel("")); // Spacer
        panelUtara_3007.add(btnTambah_3007);

        panelArray_3007 = new JPanel();
        panelArray_3007.setLayout(new FlowLayout(FlowLayout.CENTER, 10, 10));
        panelArray_3007.setBorder(BorderFactory.createTitledBorder("Visualisasi Array"));

        JScrollPane scrollVisual_3007 = new JScrollPane(panelArray_3007);
        scrollVisual_3007.setPreferredSize(new Dimension(500, 150));

        stepArea_3007 = new JTextArea(15, 30);
        stepArea_3007.setEditable(false);
        stepArea_3007.setFont(new Font("Monospaced", Font.PLAIN, 13));
        JScrollPane scrollText_3007 = new JScrollPane(stepArea_3007);
        scrollText_3007.setBorder(BorderFactory.createTitledBorder("Log Langkah Sorting"));

        JPanel panelKontrol_3007 = new JPanel(new FlowLayout());

        String[] pilihanAlgoritma_3007 = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
        comboSort_3007 = new JComboBox<>(pilihanAlgoritma_3007);

        btnMulaiSort_3007 = new JButton("Mulai Sorting");
        btnLangkah_3007 = new JButton("Langkah Selanjutnya");
        btnLangkah_3007.setEnabled(false);
        btnReset_3007 = new JButton("Reset Data");
    }
}

```

```

panelKontrol_3007.add(new JLabel("Pilih Algoritma:"));
panelKontrol_3007.add(comboSort_3007);
panelKontrol_3007.add(btnMulaiSort_3007);
panelKontrol_3007.add(btnLangkah_3007);
panelKontrol_3007.add(btnReset_3007);

// Menambahkan panel ke frame
add(panelUtara_3007, BorderLayout.NORTH);
add(scrollVisual_3007, BorderLayout.CENTER);
add(scrollText_3007, BorderLayout.EAST);
add(panelKontrol_3007, BorderLayout.SOUTH);

btnTambah_3007.addActionListener(e -> tambahData_3007());
btnMulaiSort_3007.addActionListener(e -> inisialisasiSorting_3007());
btnLangkah_3007.addActionListener(e -> performStep_3007());
btnReset_3007.addActionListener(e -> resetSemua_3007());
}

private void tambahData_3007() {
    String nama_3007 = inputNama_3007.getText().trim();
    String nim_3007 = inputNim_3007.getText().trim();
    String prodi_3007 = inputProdi_3007.getText().trim();

    if (nama_3007.isEmpty() || nim_3007.isEmpty() || prodi_3007.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Lengkapi semua field input", "Peringatan",
JOptionPane.WARNING_MESSAGE);
        return;
    }

    dataAwal_3007.add(new Mahasiswa_2511533007(nama_3007, nim_3007, prodi_3007));
    renderArrayAwal_3007();
    stepArea_3007.append("Ditambahkan: " + nama_3007 + "\n");

    inputNama_3007.setText("");
    inputNim_3007.setText("");
    inputProdi_3007.setText("");
    inputNama_3007.requestFocus();
}

private void renderArrayAwal_3007() {
    panelArray_3007.removeAll();
    labelArray_3007 = new JLabel[dataAwal_3007.size()];

    for (int k_3007 = 0; k_3007 < dataAwal_3007.size(); k_3007++) {
        labelArray_3007[k_3007] = new JLabel(dataAwal_3007.get(k_3007).getNama_3007());
        labelArray_3007[k_3007].setFont(new Font("Arial", Font.BOLD, 18));
        labelArray_3007[k_3007].setBorder(BorderFactory.createLineBorder(Color.BLACK, 2));
        labelArray_3007[k_3007].setPreferredSize(new Dimension(100, 50));
        labelArray_3007[k_3007].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_3007.add(labelArray_3007[k_3007]);
    }
    panelArray_3007.revalidate();
    panelArray_3007.repaint();
}

private void inisialisasiSorting_3007() {
    if (dataAwal_3007.size() < 2) {
        JOptionPane.showMessageDialog(this, "Minimal 2 data untuk melakukan sorting",
"Peringatan", JOptionPane.WARNING_MESSAGE);
        return;
    }

    dataProses_3007 = new ArrayList<>(dataAwal_3007);
    tipeSort_3007 = (String) comboSort_3007.getSelectedItem();

    stepCount_3007 = 1;
    sorting_3007 = true;

    if (tipeSort_3007.equals("Insertion Sort")) {
        i_3007 = 1;
    } else {
        i_3007 = 0;
    }
}

```



```

        stepArea_3007.setText("=== " + tipeSort_3007.toUpperCase() + " ===\n");
        stepArea_3007.append("Data Awal: " + arrayToString_3007() + "\n\n");
        System.out.println("=== " + tipeSort_3007.toUpperCase() + " ===");

        btnMulaiSort_3007.setEnabled(false);
        btnTambah_3007.setEnabled(false);
        btnLangkah_3007.setEnabled(true);
    }

    // Method memanggil algoritma sorting
    private void performStep_3007() {
        if (!sorting_3007) return;

        if (tipeSort_3007.equals("Insertion Sort")) {
            stepInsertionSort_3007();
        } else if (tipeSort_3007.equals("Selection Sort")) {
            stepSelectionSort_3007();
        } else if (tipeSort_3007.equals("Bubble Sort")) {
            stepBubbleSort_3007();
        }
    }

    // INSERTION SORT
    private void stepInsertionSort_3007() {
        if (i_3007 < dataProses_3007.size()) {
            Mahasiswa_2511533007 key_3007 = dataProses_3007.get(i_3007);
            int j_3007 = i_3007 - 1;

            while (j_3007 >= 0 &&
dataProses_3007.get(j_3007).getNama_3007().compareToIgnoreCase(key_3007.getNama_3007()) > 0) {
                dataProses_3007.set(j_3007 + 1, dataProses_3007.get(j_3007));
                j_3007--;
            }
            dataProses_3007.set(j_3007 + 1, key_3007);

            catatLangkah_3007("Langkah");
            i_3007++;

            if (i_3007 == dataProses_3007.size()) sortingSelesai_3007();
        }
    }

    // SELECTION SORT
    private void stepSelectionSort_3007() {
        if (i_3007 < dataProses_3007.size() - 1) {
            int minIndex_3007 = i_3007;
            for (int j_3007 = i_3007 + 1; j_3007 < dataProses_3007.size(); j_3007++) {
                if
(dataProses_3007.get(j_3007).getNama_3007().compareToIgnoreCase(dataProses_3007.get(minIndex_3007).get
Nama_3007()) < 0) {
                    minIndex_3007 = j_3007;
                }
            }
            Mahasiswa_2511533007 temp_3007 = dataProses_3007.get(i_3007);
            dataProses_3007.set(i_3007, dataProses_3007.get(minIndex_3007));
            dataProses_3007.set(minIndex_3007, temp_3007);

            catatLangkah_3007("Pass");
            i_3007++;

            if (i_3007 == dataProses_3007.size() - 1) sortingSelesai_3007();
        }
    }

    // BUBBLE SORT
    private void stepBubbleSort_3007() {
        if (i_3007 < dataProses_3007.size() - 1) {
            boolean swapped_3007 = false;

            for (int j_3007 = 0; j_3007 < dataProses_3007.size() - i_3007 - 1; j_3007++) {
                if
(dataProses_3007.get(j_3007).getNama_3007().compareToIgnoreCase(dataProses_3007.get(j_3007 +
1).getNama_3007()) > 0) {
                    // Swap

```

```

        Mahasiswa_2511533007 temp_3007 = dataProses_3007.get(j_3007);
        dataProses_3007.set(j_3007, dataProses_3007.get(j_3007 + 1));
        dataProses_3007.set(j_3007 + 1, temp_3007);
        swapped_3007 = true;
    }
}

catatLangkah_3007("Pass");
i_3007++;
if (!swapped_3007 || i_3007 == dataProses_3007.size() - 1) {
    sortingSelesai_3007();
}
}

// Memperbarui visualisasi dan log langkah
private void catatLangkah_3007(String istilahLangkah) {
    updateLabelVisual_3007();
    String hasilLangkah = istilahLangkah + " " + stepCount_3007 + " : " + arrayToString_3007();
    stepArea_3007.append(hasilLangkah + "\n");
    System.out.println(hasilLangkah);
    stepCount_3007++;
}

private void updateLabelVisual_3007() {
    for (int k_3007 = 0; k_3007 < dataProses_3007.size(); k_3007++) {
        labelArray_3007[k_3007].setText(dataProses_3007.get(k_3007).getNama_3007());
    }
    panelArray_3007.revalidate();
    panelArray_3007.repaint();
}

private void sortingSelesai_3007() {
    sorting_3007 = false;
    btnLangkah_3007.setEnabled(false);
    btnMulaiSort_3007.setEnabled(true);
    btnTambah_3007.setEnabled(true);

    stepArea_3007.append("\nPengurutan Selesai!\n");
    JOptionPane.showMessageDialog(this, "Proses Sorting Selesai!");
}

// Reset data
private void resetSemua_3007() {
    dataAwal_3007.clear();
    panelArray_3007.removeAll();
    panelArray_3007.revalidate();
    panelArray_3007.repaint();
    stepArea_3007.setText("");

    btnLangkah_3007.setEnabled(false);
    btnMulaiSort_3007.setEnabled(true);
    btnTambah_3007.setEnabled(true);

    sorting_3007 = false;
}

private String arrayToString_3007() {
    StringBuilder sb_3007 = new StringBuilder();
    sb_3007.append("[");
    for (int k_3007 = 0; k_3007 < dataProses_3007.size(); k_3007++) {
        sb_3007.append(dataProses_3007.get(k_3007).getNama_3007());
        if (k_3007 < dataProses_3007.size() - 1) sb_3007.append(", ");
    }
    sb_3007.append("]");
    return sb_3007.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        GUIMahasiswa_2511533007 gui_3007 = new GUIMahasiswa_2511533007();
        gui_3007.setVisible(true);
    });
}
}

```

C. Penjelasan Kode

Class ini adalah class utama GUI untuk memvisualisasikan tiga jenis algoritma sorting sekaligus berdasarkan abjad nama.

Konstruktor class ini membangun seluruh tampilan aplikasi yang meliputi panel input data mahasiswa, area visualisasi kotak nama, menu pilihan algoritma, tombol kendali, dan panel khusus pencatatan log.

Method `tambahData_3007` berfungsi mengambil data dari tiga kolom input, memvalidasinya agar tidak kosong, membuat objek mahasiswa baru untuk dimasukkan ke dalam daftar, lalu memicu pembaruan tampilan visual melalui method `renderArrayAwal_3007` yang menggambar ulang seluruh kotak visualisasi nama di panel utama.

Method `inisialisasiSorting_3007` memvalidasi kesiapan jumlah data, menyalin daftar data awal ke daftar proses agar data asli tidak rusak, menentukan batas awal indeks perulangan sesuai algoritma yang dipilih, serta mengaktifkan tombol kendali langkah.

Saat tombol ditekan, method `performStep_3007` bertindak sebagai pengatur percabangan yang memanggil metode pengurutan yang spesifik.

Method `stepInsertionSort_3007` melakukan satu tahap pengurutan Insertion Sort pada daftar mahasiswa dengan membandingkan teks nama menggunakan fungsi perbandingan abjad, lalu menggeser elemen objek jika posisinya belum tepat.

Method `stepSelectionSort_3007` melakukan satu tahapan penuh Selection Sort untuk mencari objek mahasiswa dengan abjad nama terkecil dari sisa barisan lalu menukarnya ke posisi indeks acuan.

Method `stepBubbleSort_3007` melakukan satu tahapan Bubble Sort dengan menyisir seluruh barisan data dari awal, membandingkan nama yang bersebelahan, dan menukarnya jika tidak berurutan.

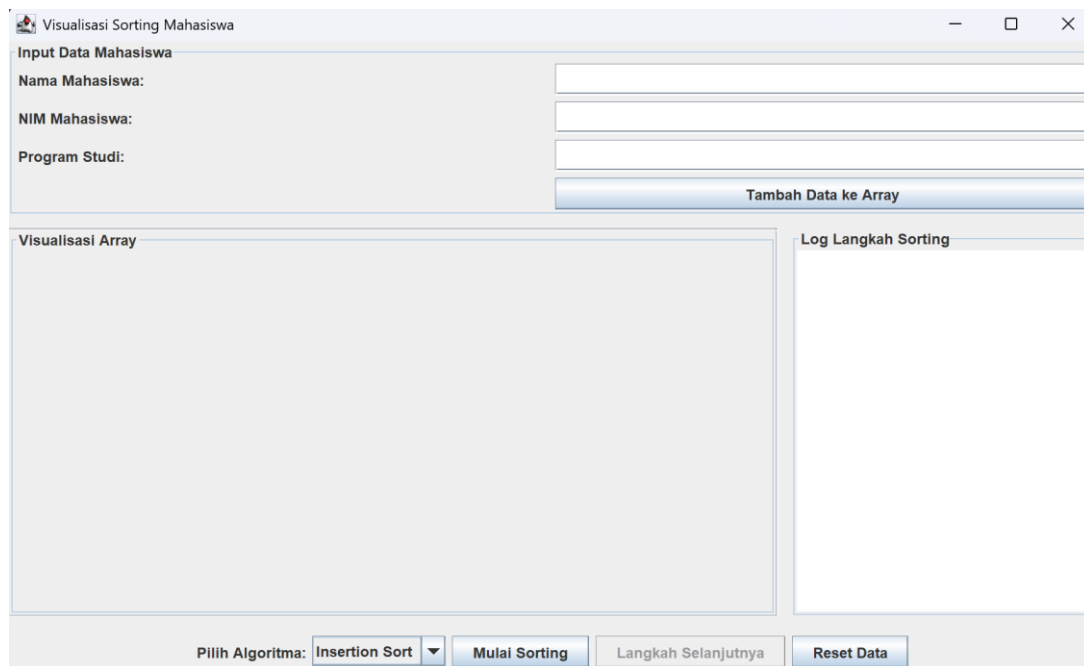
Method `catatLangkah_3007` mengonversi urutan data terbaru menjadi teks informasi untuk ditampilkan ke area log aplikasi melalui bantuan method `updateLabelVisual_3007` yang memperbarui urutan teks nama pada komponen visual secara berkala.

Jika pengurutan selesai, method `sortingSelesai_3007` akan mengembalikan status aplikasi ke mode normal dan menampilkan kotak dialog informasi sukses.

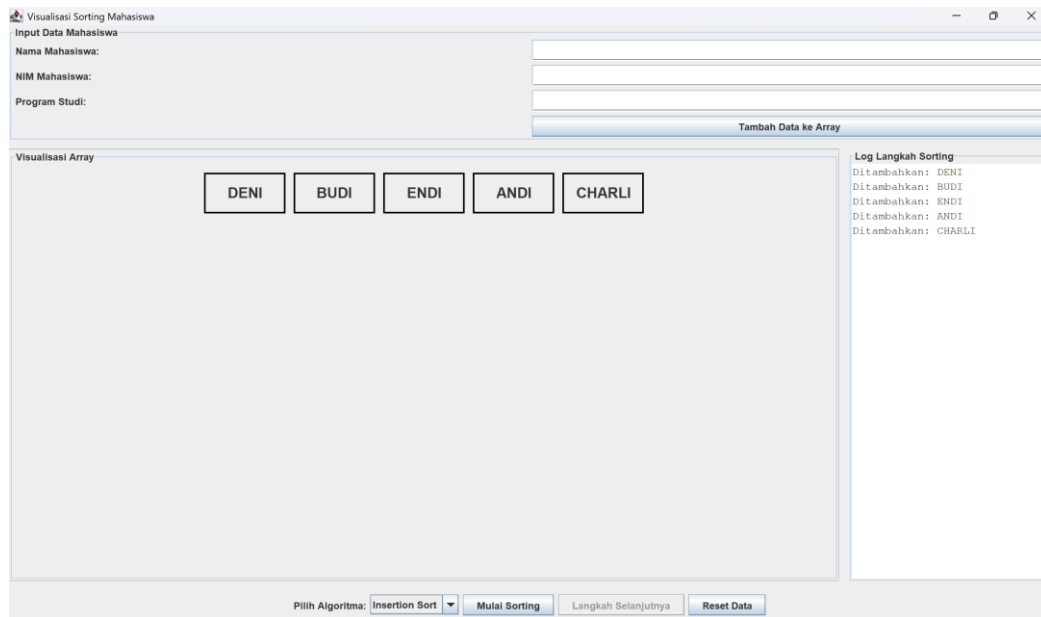
Method `resetSemua_3007` digunakan untuk menghapus seluruh data mahasiswa dari memori serta membersihkan seluruh komponen visual dan teks log di layar.

Method `arrayToString_3007` menggabungkan kumpulan nama mahasiswa di dalam daftar proses menjadi satu teks string utuh berformat tanda kurung siku demi kebutuhan pencatatan log, dan method `main` menjadi pengesekusi utama untuk menampilkan aplikasi visualisasi mahasiswa terpadu ini di layar pengguna.

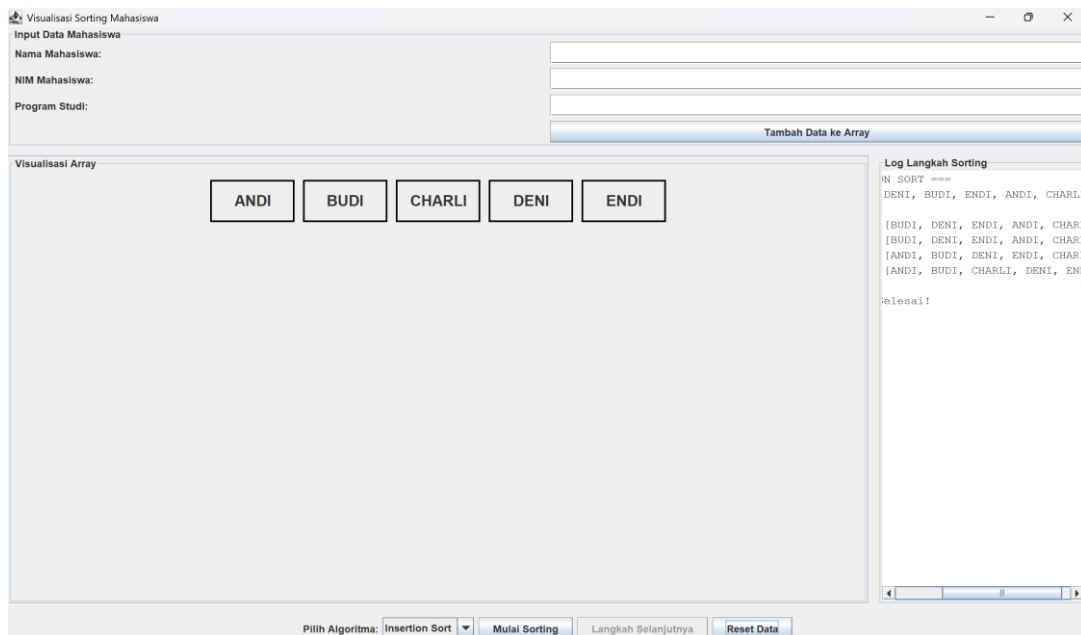
D. Output Kode



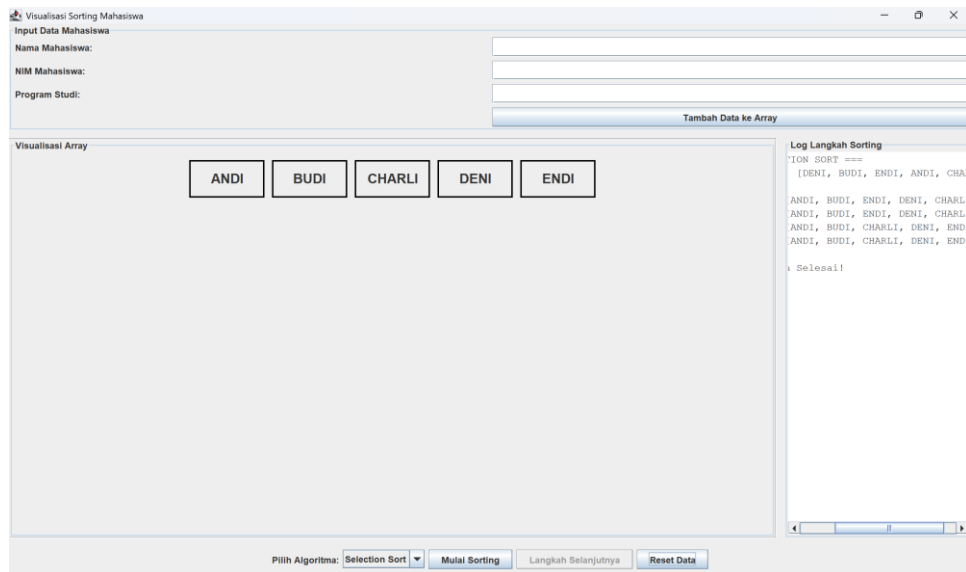
Output 1 tampilan output



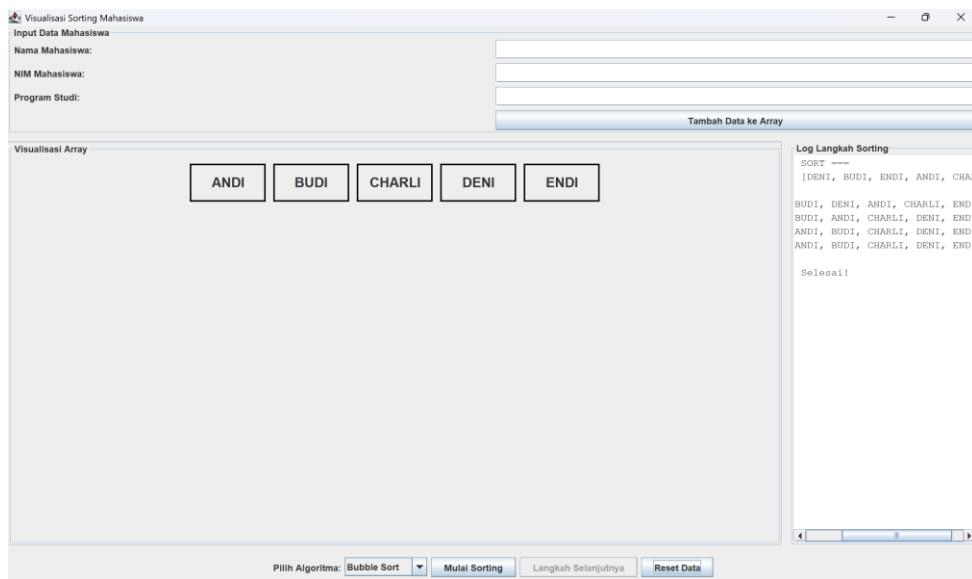
Output 2 memasukkan data



Output 3 proses insertion sort



Output 4 proses selection sort



Output 5 proses bubble sort